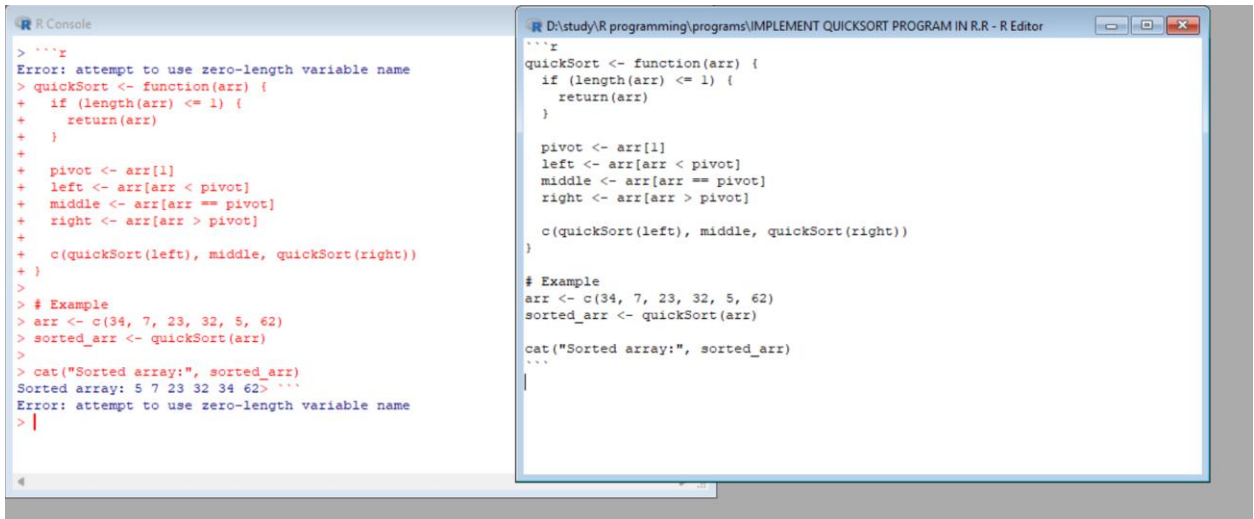


1. IMPLEMENT QUICKSORT PROGRAM IN R



The screenshot shows two windows from an R environment. The left window is the 'R Console' and the right is the 'R Editor'.

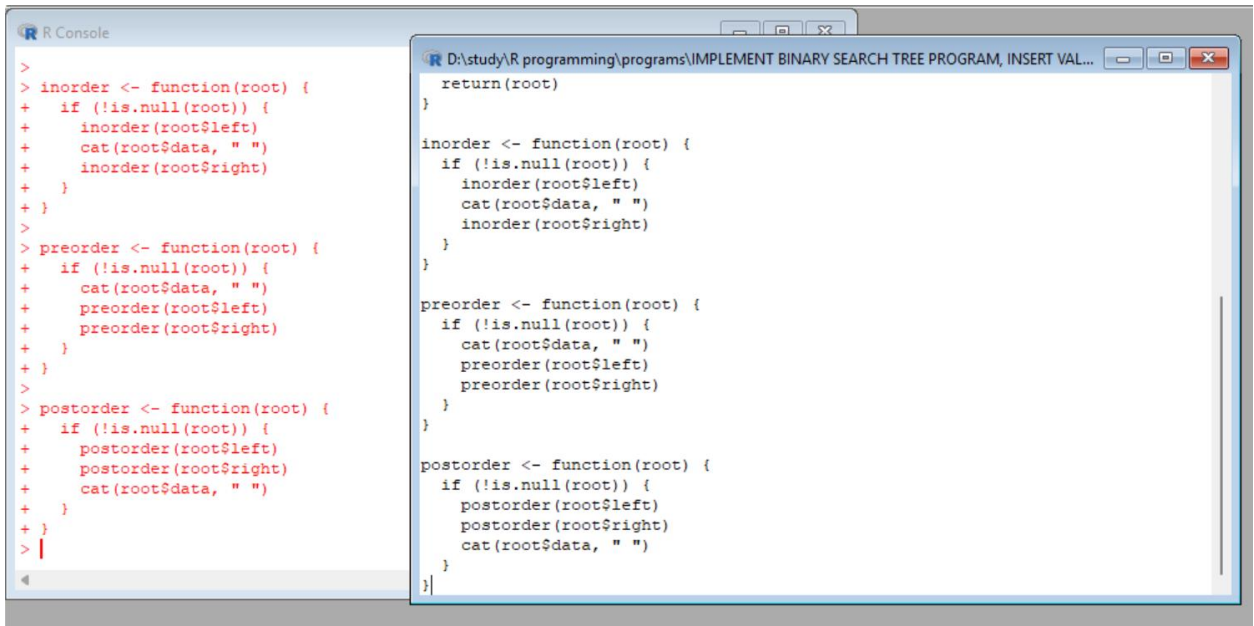
R Console:

```
> '''  
Error: attempt to use zero-length variable name  
> quickSort <- function(arr) {  
+   if (length(arr) <= 1) {  
+     return(arr)  
+   }  
+  
+   pivot <- arr[1]  
+   left <- arr[arr < pivot]  
+   middle <- arr[arr == pivot]  
+   right <- arr[arr > pivot]  
+  
+   c(quickSort(left), middle, quickSort(right))  
+ }  
>  
> # Example  
> arr <- c(34, 7, 23, 32, 5, 62)  
> sorted_arr <- quickSort(arr)  
>  
> cat("Sorted array:", sorted_arr)  
Sorted array: 5 7 23 32 34 62  
Error: attempt to use zero-length variable name  
> |
```

R Editor:

```
'''  
quickSort <- function(arr) {  
  if (length(arr) <= 1) {  
    return(arr)  
  }  
  
  pivot <- arr[1]  
  left <- arr[arr < pivot]  
  middle <- arr[arr == pivot]  
  right <- arr[arr > pivot]  
  
  c(quickSort(left), middle, quickSort(right))  
}  
  
# Example  
arr <- c(34, 7, 23, 32, 5, 62)  
sorted_arr <- quickSort(arr)  
  
cat("Sorted array:", sorted_arr)  
'''  
|
```

2. IMPLEMENT BINARY SEARCH TREE PROGRAM, INSERT VALUES, AND DISPLAY ITS VALUES IN ALL TRAVERSAL FORM, INORDER, PREORDER, POSTORDER.



The screenshot shows two windows from an R environment. The left window is the 'R Console' and the right is the 'R Editor'.

R Console:

```
>  
> inorder <- function(root) {  
+   if (!is.null(root)) {  
+     inorder(root$left)  
+     cat(root$data, " ")  
+     inorder(root$right)  
+   }  
+ }  
>  
> preorder <- function(root) {  
+   if (!is.null(root)) {  
+     cat(root$data, " ")  
+     preorder(root$left)  
+     preorder(root$right)  
+   }  
+ }  
>  
> postorder <- function(root) {  
+   if (!is.null(root)) {  
+     postorder(root$left)  
+     postorder(root$right)  
+     cat(root$data, " ")  
+   }  
+ }  
> |
```

R Editor:

```
return(root)  
}  
  
inorder <- function(root) {  
  if (!is.null(root)) {  
    inorder(root$left)  
    cat(root$data, " ")  
    inorder(root$right)  
  }  
}  
  
preorder <- function(root) {  
  if (!is.null(root)) {  
    cat(root$data, " ")  
    preorder(root$left)  
    preorder(root$right)  
  }  
}  
  
postorder <- function(root) {  
  if (!is.null(root)) {  
    postorder(root$left)  
    postorder(root$right)  
    cat(root$data, " ")  
  }  
}  
|
```

3. EXPLAIN APPLY, LAPPLY, TAPPLY, SAPPLY. COMPARE AND CONTRAST WITH EXAMPLES.

```
R Console
>
>
> # sapply()
> cat("\nsapply():\n")

sapply():
> print(sapply(lst, sum))
  a  b
6 15
>
>
> # tapply()
>
> marks <- c(80, 90, 75, 85, 95, 70)
> group <- c("A", "A", "B", "B", "A", "B")
>
> cat("\ntapply():\n")

tapply():
> print(tapply(marks, group, mean))
      A      B
88.33333 76.66667
>

D:\study\R programming\programs\EXPLAIN APPLY, LAPPLY, TAPPLY, SAPPLY. COMPARE AND CONTRAST WITH EXAMPLES.R
|
cat("\nColumn Sum:\n")
print(apply(mat, 2, sum))

# lapply()

lst <- list(a = 1:3, b = 4:6)

cat("\nlapply():\n")
print(lapply(lst, sum))

# sapply()

cat("\nsapply():\n")
print(sapply(lst, sum))

# tapply()

marks <- c(80, 90, 75, 85, 95, 70)
group <- c("A", "A", "B", "B", "A", "B")

cat("\ntapply():\n")
print(tapply(marks, group, mean))
```